Eng 62: Module 6: Embedded Networking

DO NOT TURN IN THIS LAB SHEET

Write your name here:

Module 6 Learning Objectives. To become familiar with:

1. Event Driven Programming
2. Finite State Machine (FSM) Design
3. Realization of control systems using Event driven FSM's

TASKS

Complete each of the following tasks and obtain a TA signature for each task:

STEP 1. Introduction. Review and make notes on the introductory video M 6.0.

STEP 2. [NOTE: This is a paper and pencil exercise]. Design a **finite state machine** that controls the *railway crossing.* The crossing consists of a one-way road with an associated set of traffic lights, a blue maintenance light, a railway track with a gate, and a pedestrian crossing with two associated push buttons. Each crossing has a unique id.

Generally, when there are no trains, traffic is allowed to flow continuously for a *minimum* of three minutes. If a pedestrian pushes a crossing button, only after the three minutes are up and the red light is illuminated, pedestrians are given 30 seconds to cross the road. The traffic lights sequence on a 3 second interval between different colors.

The crossing communicates with a *railway switching substation* that holds a database of *crossings* and their *status*. When a train is arriving, the substation sends an "train arriving" message to the controller. The controller stops traffic flow and closes the gate. The substation monitors the railway line and informs the controller when the train has passed with a "clear" message; the controller then raises the gate, waits 20 seconds to ensure there are no pedestrians on the crosswalk, and allows traffic to flow with the appropriate traffic signals. While the traffic is stopped, pedestrians can freely cross the road.

Occasionally, a railway engineer may walk up to the crossing, insert a key, and manually close the crossing for maintenance; in this mode trains may still pass, but the engineer may work on the crossing hardware without traffic flowing. When the key is turned, the controller stops the traffic, closes the gate, and flashes a blue light on the gate at 1 second intervals. A wheel at the side of the road can be used to manually raise and lower the gate. The engineer may manually initiate opening the crossing by use of his key.

(5 points) TA Signature:

STEP 3. Implement your finite state machine using the following:

- Use RGB (and yellow) LEDs to represent the traffic signals and maintenance light. Lights should normally sequence red-yellow-green-yellow-red. The traffic lights are *off* in maintenance mode.
- Use the Servo motor to control the gate with a *full* 90-degree swing (up/down).
- Use switches to indicate Maintenance Mode/Clear and Train Arrival /Clear.
- Use buttons for the pedestrian signal – one on each side of the road.
- Use 10 seconds instead of the 3 minutes for the traffic flow minimum
- Use 10 seconds instead of the 20 seconds for the pedestrian walk time.
- Use the potentiometer for the wheel at the side of the road.
- Signify the gate status (open/closed), maintenance mode (entry/exit), and train status (arriving/clear) using a message on the screen.
- Turn on LEDs to indicate to pedestrians that they may cross.
- Time all periods to within $1/10^{th}$ of a second.

(12 points) TA Signature:

STEP 4. On canvas you will find a program *substation.c* that is a web-client that can be used to update status entries in the substation database, or ping the substation to ensure it is alive. Do not include it in your Vivado project, but instead simply compile it from the Linux command line using:

```
gcc substation.c -o substation
```

You can then run the code with:

```
./substation <id>      -- where id is your class id from Module 4
```

This will allow you to set the train status entry associated with your crossing controller (identified by your id) on the substation server.

Using the Wifi module, poll the "remote substation status database" 10 times per second to discover the train status, using an UPDATE message (as per Module 5), with id 0; Note that the substation responds (without setting the value associate with your id) with the status of all the crossings (ids), from which you can extract the status.

Add the capability for the substation to initiate and terminate maintenance mode remotely, *without* the operator using a key.

(3 points) TA Signature: